

Đại học Quốc gia Thành phố Hồ Chí Minh

Đại học Khoa học tự nhiên

Khoa Công nghệ thông tin



HOMework

DATA GENERATION & SCENARIO TESTIN

Subject **SOFTWARE TESTING**

Class **22KTPM3**

Student **22127374 – Lê Thanh Tâm**

Ho Chi Minh City, 2025

Table of Contents

1	Group Task Allocation	2
2	Introduction	2
2.1	Selected Features for Testing	2
2.2	Testing Methodology - Scenario Testing	2
3	Feature 1: Admin - Orders	3
3.1	Feature Description	3
3.2	Flow and Scenario Design	4
3.3	Test Case Documentation	4
4	Feature 2: Admin - Categories	5
4.1	Feature Description	5
4.2	Flow and Scenario Design	5
4.3	Test Case Documentation	6
5	Feature 3: Admin - Products	6
5.1	Feature Description	6
5.2	Flow and Scenario Design	7
5.3	Test Case Documentation	8
6	Merged Test Case List	8
6.1	Consolidation of AI and Manual Test Cases	8
6.2	Deduplication Process and Justification	8
7	Use of AI Tools	9
7.1	AI Tool Description	9
7.2	Prompts Used	9
7.3	Validation and Refinement Process	11
7.4	Breakdown of AI vs. Manual Test Cases	11
8	Test Execution and Bug Reporting	11
8.1	Test Execution Summary	11
8.2	Bug Report and Analysis	12
9	Self-Assessment	13

1 Group Task Allocation

Fullname	22127374	Feature
Lương Hoàng Dung	22127076	1. User - Product Browsing Experience 2. User - Contact Form Submission 3. User - Checkout
Lê Thanh Tâm	22127374	1. Admin - Orders 2. Admin - Categories 3. Admin - Products
Nguyễn Lâm Nhã Uyên	22127445	1. Log in 2. User - Sign up
Lê Nguyễn Yến Vy	22127466	1. User - add to cart and quantity management 2. User - change password

2 Introduction

2.1 Selected Features for Testing

The features selected for this assignment (Scenario Testing) are a direct continuation of the work completed in the previous homework on **Domain Testing**. The same three high-priority administrative features, previously analyzed for their data domains and business rules, have been chosen for workflow testing:

- **Feature 1 (UC01):** Admin - Orders
- **Feature 2 (UC02):** Admin - Categories
- **Feature 3 (UC03):** Admin - Products

This approach was chosen strategically. The in-depth analysis of input fields and data constraints from the Domain Testing exercise provides the foundational knowledge required to design meaningful and realistic end-to-end user scenarios. While the previous assignment focused on data-level validation, this report builds upon that understanding to test the complete user workflows and interactions within these critical features.

2.2 Testing Methodology - Scenario Testing

The primary methodology applied is **Scenario Testing**, as required by the assignment. Scenario testing is a black-box test design technique where test cases are designed to execute use-case scenarios.

To systematically derive these scenarios, the principles of **Use Case Testing** are employed. For this report, each "Feature" is treated as a "Use Case." A Use Case is a list of action steps that define the interactions between an actor and the system to achieve a specific goal. Each Use Case is analyzed using the following concepts:

- **Basic Flow:** This describes the normal, "happy path" sequence of events when the use case is performed successfully from start to finish.

- **Alternate Flows:** These describe optional or exceptional behaviors that deviate from the normal path.
- **Scenario:** A scenario represents a complete "path" through the use case, formed by combining the Basic Flow with zero or more Alternate Flows.

3 Feature 1: Admin - Orders

3.1 Feature Description

The **Admin - Orders** feature provides a comprehensive interface within the Admin panel for managing and processing orders. It enables administrators to search, view, and update order details efficiently while ensuring robust validation and security measures. Key functionalities include:

- **Search Capabilities:** Supports searching orders by invoice number (valid/existing or non-existing), billing address (partial or exact matches), invoice date (specific dates, current date, or full datetime strings), and status filters via a dropdown menu. Handles edge cases such as empty strings, very short or long search terms (up to 255 characters), special characters, case sensitivity, and leading/trailing whitespace.
- **Order Status Management:** Enforces a sequential order status workflow (AWAITING_FULFILLMENT → ON_HOLD → AWAITING_SHIPMENT → SHIPPED → COMPLETED), preventing reversion to previous statuses or non-sequential updates. Supports manual status updates via a dropdown menu, automatic status updates upon successful customer payment, and validation to ensure only valid status transitions are allowed.
- **Status Message Handling:** Allows administrators to save meaningful status messages (up to 255 characters) with robust input validation. Rejects or sanitizes special characters and HTML/script tags to prevent Cross-Site Scripting (XSS) vulnerabilities. Handles edge cases like empty inputs, whitespace-only inputs (treated as empty after trimming), and messages at or beyond the maximum length limit.
- **Data Integrity and Security:** Ensures non-editable fields (e.g., Invoice Number, Total, Billing Address) remain protected during order edits. Validates inputs to prevent invalid data (e.g., incorrect tracking numbers) and maintains synchronization between Payment and Order Management subsystems for accurate status updates.
- **User Interface and Functionality:** Provides navigation to the order edit interface, a reset button to clear search filters, and error handling for invalid inputs or actions (e.g., attempting to update to an unchanged status). Ensures all interactions are secure, user-friendly, and compliant with system constraints.

This feature is critical for streamlined order management, ensuring data accuracy, secure operations, and seamless integration with payment processing, as validated through 30 test cases covering search, status updates, input validation, and system synchronization.

3.2 Flow and Scenario Design

Basic Flow:

1. Admin navigates to the "Admin Orders"page. The page displays a list of orders.
2. Admin clicks the **Edit** button for a specific order (e.g., one with status "AWAITING_FULFILLMENT").
3. The system navigates to the order detail/edit page.
4. Admin changes the "Status"field to a new valid status (e.g., "SHIPPED").
5. Admin clicks a "Save"or "Update"button on the detail page.
6. The system returns to the order list, showing a success message and the updated status for that order.

Alternate Flows:

- **AF1: Search Order:** At step 1, Admin enters text (e.g., an Invoice Number) into the search box and clicks the **Search** button. The list is filtered accordingly.
- **AF2: Reset Search:** After performing a search, Admin clicks the **Reset** button to restore the full list of orders.
- **AF3: Invalid Status Update:** At step 4, the Admin attempts an invalid status transition (e.g., updating a "COMPLETED"order back to "AWAITING_FULFILLMENT").

Scenarios:

Scenario ID	Description	Path (Flow Combination)
S1	Successful Order Update	Basic Flow
S2	Invalid Input (Search)	Basic Flow (step 1) + AF1 (with invalid data)
S3	Search and Reset	Basic Flow (step 1) + AF1 + AF2
S4	Failed Status Update	Basic Flow (up to step 4) + AF3

3.3 Test Case Documentation

As an exercise in **Scenario Testing**, a comprehensive set of test cases was designed for the **Admin - Orders** feature. These test cases were systematically derived from the scenarios defined in the previous section, which are combinations of the Basic and Alternate Flows. The primary focus is on:

- **Basic Flow Scenarios:** To validate the end-to-end "happy path"functionality, ensuring a user can successfully complete the primary goal of the use case without any errors.
- **Alternate Flow Scenarios:** To test the system's robustness and error-handling capabilities. This includes testing with invalid inputs, handling system exceptions, and validating optional user paths.

The detailed documentation for each test case including the Test Case ID, Description, Preconditions, Steps, and Expected Results has been carefully recorded. For a complete and precise understanding, please access and review the Test Cases sheet in the file **22127374_TestCases.xlsx**.

4 Feature 2: Admin - Categories

4.1 Feature Description

The **Admin - Categories** feature enables management of product categories with:

- **Add/Edit:** Create/edit top-level/sub-categories with valid Name (up to 120 characters), Slug, and Parent ID. Rejects blank/duplicate Names, emojis, invalid Slugs, or Names over 120 characters. ID is display-only.
- **Delete:** Delete leaf-node/top-level categories not used as Parent IDs; blocks deletion of parent categories.
- **Search:** Search by full/partial Name, single character, or emoji; handles non-matching strings and resets via Reset button.
- **Navigation:** Back button discards changes on Add/Edit forms, returning to the Categories list.
- **Frontend Display:** Top-level categories (e.g., Hand Tools) appear in navigation; clicking leads to correct pages with accurate sub-category filters.
- **Sorting/Filtering:** Sort products by Name (A-Z/Z-A) or Price (High-Low/Low-High) on Hand Tools/Power Tools pages. Filter by sub-category (e.g., Pliers, Sander), brand, or both; clear filters to reset. Ensures efficient category management and accurate product organization.

4.2 Flow and Scenario Design

Basic Flow:

1. Admin navigates to the "Admin Categories" page.
2. Click the **Add Category** button.
3. The system displays a form for adding a new category.
4. Admin fills in the "Name" and "Slug" fields with unique, valid data.
5. Admin clicks the "Save" or "Add Category" button on the form.
6. The system returns to the categories list, displays a success message, and the new category is visible in the table.

Alternate Flows:

- **AF1: Invalid Data Input:** At step 4, enter invalid data (e.g., blank Name, duplicate Slug).
- **AF2: Edit Category:** At step 1, click the **Edit** button on an existing category.
- **AF3: Delete Category:** At step 1, click the **Delete** button on a category. This flow considers deleting a category with and without dependencies (sub-categories).
- **AF4: Search/Reset:** At step 1, use the Search and Reset buttons to filter the list.

- **AF5: Use Back Button:** After clicking "Add Category" or "Edit", use the back functionality to cancel the operation.

Scenarios:

Scenario ID	Description	Path (Flow Combination)
S1	Successful Add Category	Basic Flow
S2	Invalid Add Category	Basic Flow (up to step 4) + AF1
S3	Successful Edit Category	Basic Flow (step 1) + AF2
S4	Delete Category	Basic Flow (step 1) + AF3
S5	Search and Reset	Basic Flow (step 1) + AF4
S6	Sort and Filter	Basic Flow (step 1) + AF5
S7	UI Navigation	Basic Flow (step 1) + AF6
S8	Back Button Functionality	Basic Flow (up to step 2) + AF7

4.3 Test Case Documentation

As an exercise in **Scenario Testing**, a comprehensive set of test cases was designed for the **Admin - Categories** feature. These test cases were systematically derived from the scenarios defined in the previous section, which are combinations of the Basic and Alternate Flows. The primary focus is on:

- **Basic Flow Scenarios:** To validate the end-to-end "happy path" functionality, ensuring a user can successfully complete the primary goal of the use case without any errors.
- **Alternate Flow Scenarios:** To test the system's robustness and error-handling capabilities. This includes testing with invalid inputs, handling system exceptions, and validating optional user paths.

The detailed documentation for each test case—including the Test Case ID, Description, Preconditions, Steps, and Expected Results—has been carefully recorded. For a complete and precise understanding, please access and review the Test Cases sheet in the file **22127374_TestCases.xlsx**.

5 Feature 3: Admin - Products

5.1 Feature Description

The **Admin - Products** feature enables administrators to manage products with the following functionalities:

- **Search:** Search by full/partial product name or stock quantity, handling empty strings, spaces, or long inputs. Displays "No products found" for no matches.
- **Add Product:** Add products with mandatory (Name, Price, Brand) and optional (Description, Category) fields. Supports names up to 220 characters, valid numeric Price/Stock, and handles invalid inputs (e.g., blank Name, non-numeric Price).
- **Edit Product:** Update fields with valid data, allow blank Description, prevent editing ID or invalid inputs, and permit duplicate names.

- **Delete Product:** Supports soft delete with confirmation, prevents deletion of products with dependencies (e.g., in orders), and allows canceling deletion.
- **Pagination:** Navigate via Next, Previous, or specific page numbers, maintaining state after searches, with correct first/last page display.
- **Reset/Back:** Reset button restores the product list; Back button discards changes or returns to the list.

Ensures efficient product management, input validation, and user-friendly pagination.

5.2 Flow and Scenario Design

Basic Flow:

1. Admin navigates to the "Admin Products" page, where a paginated list of products is displayed.
2. Click the **Add Product** button.
3. The system displays a form to add a new product.
4. Admin fills in all required fields (e.g., Name, Stock, Price) with valid data.
5. Click the "Save" button on the form.
6. The system returns to the product list, displays a success message, and the new product is visible.

Alternate Flows:

- **AF1: Invalid Data Input:** At step 4, Admin enters invalid data (e.g., non-numeric price).
- **AF2: Search Product:** At step 1, Admin uses the **Search** button to find a product.
- **AF3: Edit Product:** At step 1, Admin clicks the **Edit** button for an existing product.
- **AF4: Delete Product:** At step 1, Admin clicks the **Delete** button for a product.
- **AF5: Use Pagination:** At step 1, Admin clicks on a page number or the **Next** link to navigate to another page of products.
- **AF6: Use Reset:** After a search, Admin clicks the **Reset** button.

Scenarios:

Scenario ID	Description	Path (Flow Combination)
S1	Search Product	Basic Flow (step 1) + AF2
S2	Add Product	Basic Flow (for success) AND Basic Flow (up to step 4) + AF1 (for failure)
S3	Edit Product	Basic Flow (step 1) + AF3
S4	Delete Product	Basic Flow (step 1) + AF4
S5	Pagination	Basic Flow (step 1) + AF5
S6	Navigation/Reset	Basic Flow (step 1) + AF2 + AF6

5.3 Test Case Documentation

As an exercise in **Scenario Testing**, a comprehensive set of test cases was designed for the **Admin - Products** feature. These test cases were systematically derived from the scenarios defined in the previous section, which are combinations of the Basic and Alternate Flows. The primary focus is on:

- **Basic Flow Scenarios:** To validate the end-to-end "happy path" functionality, ensuring a user can successfully complete the primary goal of the use case without any errors.
- **Alternate Flow Scenarios:** To test the system's robustness and error-handling capabilities. This includes testing with invalid inputs, handling system exceptions, and validating optional user paths.

The detailed documentation for each test case—including the Test Case ID, Description, Preconditions, Steps, and Expected Results—has been carefully recorded. For a complete and precise understanding, please access and review the Test Cases sheet in the file `22127374_TestCases.xlsx`.

6 Merged Test Case List

The final test suite was developed through a hybrid process of AI generation and human oversight to maximize both efficiency and testing depth.

6.1 Consolidation of AI and Manual Test Cases

The process began by merging two distinct sets of test cases:

- **AI-Generated Baseline (approx. 40%):** This provided a broad and rapid coverage of foundational scenarios. The AI was tasked with generating test cases for the **Basic Flow** (happy path) and common **Alternate Flows**, such as those involving empty or invalid data inputs for each feature. This efficient baseline constituted about 40% of the initial test suite.
- **Manual Augmentation (approx. 60%):** This added critical depth and context, focusing on complex, multi-step business scenarios that require human intuition and domain expertise. These manually designed test cases cover nuanced business rules (e.g., invalid status transitions for an order) and exploratory paths that an AI might overlook.

6.2 Deduplication Process and Justification

The consolidated list was then refined to eliminate redundancy. While some test cases with similar objectives were merged, others that appeared alike were intentionally kept separate for precise validation.

- **Key Justification Example:** The test case for adding a category with a duplicate **Name** and the one for adding a category with a duplicate **Slug** were both kept.

Reasoning: Although both test cases check for duplicate data, they validate two completely independent business rules and database constraints. A failure in one (e.g., the system allows a duplicate name) does not imply a failure in the other. Therefore, testing them as separate test cases is critical to ensure both data integrity rules are correctly implemented and enforced. The final decision on merging or retaining test cases was always made by the human tester to ensure thorough coverage was not compromised.

7 Use of AI Tools

7.1 AI Tool Description

To leverage state-of-the-art technology, the project utilized the most current and powerful versions of leading Large Language Models (LLMs) available at the time of the testing phase. The specific flagship models were:

- **Gemini 2.5 Pro (Google):** The most advanced and capable version in the Gemini family publicly released by Google, known for its large context window and multimodal reasoning.
- **GPT-4o (OpenAI):** OpenAI's latest and most advanced flagship model ("o" for "omni"), which powers the premium ChatGPT service and surpasses previous iterations like GPT-4.
- **Grok-1.5 (xAI):** The most powerful and recent version of the Grok model that has been officially released and documented by xAI.

These specific versions were chosen to ensure the project benefited from the most up-to-date capabilities in logical analysis, data generation, and scenario brainstorming.

7.2 Prompts Used

A multi-stage prompting strategy was used to break down the task, moving from broad scenario generation to detailed analysis. Below are representative examples based on the **Admin - Categories** feature, along with the expected AI responses.

- **Prompt 1: Initial Scenario & Test Case Generation**

"Based on the feature description for 'Admin - Categories', design a set of test scenarios covering the Basic Flow (successful category creation) and common Alternate Flows (e.g., adding with invalid data, duplicate data). Then, generate foundational test cases for these scenarios in a table format."

AI Results:

The AI would typically generate a table with foundational test cases like the following:

TC ID	Description	Test Steps	Expected Result
(TC-AI-01)	Scenario: Basic Flow - Successful Add Category.	1. Navigate to Categories. 2. Click Add. 3. Enter valid Name and Slug. 4. Click Save.	Success message is shown. New category appears in list.
(TC-AI-02)	Scenario: Alt. Flow - Add with empty Name.	1. Navigate to Categories. 2. Click Add. 3. Leave Name blank. 4. Click Save.	Error message "Name is required" is shown.
(TC-AI-03)	Scenario: Alt. Flow - Add with duplicate Slug.	1. Navigate to Categories. 2. Click Add. 3. Enter existing slug. 4. Click Save.	Error message "Slug already exists" is shown.

- **Prompt 2: Deep-Dive into a Specific Business Logic**

"Thank you. Now, let's focus on a complex business rule. For the 'Admin - Categories' feature, design detailed test cases for the scenario where a user tries to delete a parent category that has existing sub-categories, which should be blocked by the system."

AI Results:

The AI would provide more targeted test ideas:

- **(TC-DELETE-PARENT-01) - Verify Deletion Fails:** Design a test to attempt deleting a category known to be a parent (e.g., "Hand Tools"). Precondition: "Hand Tools" has sub-categories. Expected Result: An error message "Cannot delete a category with sub-categories" is displayed.
- **(TC-DELETE-LEAF-02) - Verify Deletion Succeeds:** As a control, design a test to delete a leaf-node category (one with no children) to ensure the delete function works in valid cases.

- **Prompt 3: Generating Specific Test Data**

"For the test case that checks for an invalid 'Slug' format (TC05), I need some test data. Please generate 5 example strings for the 'Slug' field that are invalid because they contain spaces, uppercase letters, or special characters."

AI Results:

The AI would generate a list of specific data points:

- "invalid slug with spaces"
- "Invalid-Uppercase"
- "slug-with-!@# \$"
- starts-with-hyphen"
- "ends-with-hyphen-"

- **Prompt 4: Final Review for Coverage Gaps**

"This is my final consolidated list of scenarios for the 'Admin - Categories' feature. Please review it and answer two questions: 1. Are there any scenarios with overlapping objectives? 2. Based on the initial feature description, are there any obvious coverage gaps?"

AI Results:

The AI would provide analytical feedback:

- **Redundancy Check:** "Scenarios S7 (UI Navigation) and S8 (Back Button) both test user navigation. They are not redundant as S7 tests general page flow while S8 tests a specific 'cancel' action, but ensure their titles clearly state this distinction."
- **Coverage Gap Analysis:** "There appears to be a gap. The current scenarios test adding a category with a duplicate **Name**. It would be beneficial to add a new test case that attempts to **edit** an existing category's name to one that already exists, as the validation logic for the 'edit' action might be implemented differently from the 'add' action."

7.3 Validation and Refinement Process

A systematic, four-step process was implemented to ensure the quality and relevance of the final test suite.

- **Foundational Review:** The initial test cases generated by the AI were treated as a "first draft." They were immediately reviewed by the tester to correct any misinterpretations of the requirements and to align them with the project's specific terminology and context.
- **Human-led Augmentation:** Recognizing that an AI may not capture all business nuances, the initial set was expanded with manually created test cases. These new cases focused on complex, multi-step user scenarios, specific business rules not explicitly documented, and exploratory tests based on the tester's intuition and domain experience.
- **AI-Assisted Consolidation:** The combined list (AI-generated + manual) was then fed back to the AI for a final review using the second prompt. The AI's suggestions for eliminating redundancies were critically assessed. For example, if the AI suggested merging a test for an "empty string" and a "whitespace-only string," the tester made the final call on whether this was appropriate or if they needed to remain separate tests to check the `trim()` function.
- **Final Approval:** The fully refined and consolidated list was then formally approved by the tester, who holds ultimate responsibility for the scope and quality of the test suite.

7.4 Breakdown of AI vs. Manual Test Cases

The final test suite represents a hybrid model, with the following approximate breakdown:

- **AI-Generated Baseline (46%):** The majority of foundational test cases originated from the AI's output (16 out of 35 TCs). These primarily covered the standard **Basic Flows** and common **Alternate Flows** for individual features (e.g., correct/incorrect data formats, non-existent records). This provided a broad and solid foundation very quickly.
- **Manual-Created & Heavily-Modified (54%):** The remaining portion was either created from scratch or significantly modified by the tester. This includes:
 - **Complex Scenarios & Business Logic (35%):** Multi-step user journeys and tests for specific business rules that require a deeper understanding of the system's goals (e.g., "a parent category cannot be deleted," "order status must be updated sequentially").
 - **Exploratory & UI/UX Tests (19%):** Test cases for UI interactions like pagination, back buttons, and combined filtering/sorting, which are based on the tester's intuition about potential user confusion or non-obvious workflows.

8 Test Execution and Bug Reporting

8.1 Test Execution Summary

- The test execution phase was conducted on **June 23, 2025**, as indicated in the test case logs.

- All 35 test cases designed in the test suite were executed manually on the `sprint5-with-bugs` version of the project. The execution scope covered the three main features: **Admin - Orders**, **Admin - Categories**, and **Admin - Products**.
- The execution process revealed a **significant number of failures (15 out of 35 test cases failed)**. For every test case with a "FAILED" status, a corresponding defect has been logged in the detailed Bug Report file to ensure proper monitoring, prioritization, and resolution.

8.2 Bug Report and Analysis

Defect Summary The testing process has identified numerous critical defects across all three features. A large portion of these defects (8 out of 15) are classified with High priority, indicating that the quality of the current build is significantly below the acceptable threshold for release. These are not minor UI issues, but serious flaws affecting core business workflows, data integrity, and essential administrative functionalities. This situation poses a significant risk to business operations and must be considered the top priority for the development team.

Highlighted High-Priority Defects While a comprehensive list is available in the bug report file, the following defects are highlighted due to their severe impact on the system's functionality and integrity:

- **B006 (Priority: High) - Order Status Fails to Synchronize Correctly After Payment**
Description: After a customer's payment is successfully confirmed, the order status on the admin panel is not updated correctly, preventing the order from moving to the next stage in the fulfillment workflow.
Impact: This breaks the core automated order processing workflow, creating a high risk of overlooking paid orders and causing significant operational delays.
- **B012 (Priority: High) - Core Product Sorting and Filtering is Non-Operational**
Description: The primary functionalities to sort products (by name) and to filter them by sub-category on public-facing category pages are completely broken.
Impact: This severely degrades the customer's shopping experience, making it impossible for them to find products efficiently and likely leading to lost sales.
- **B010 & B015 (Priority: High) - Critical Data Integrity Gap: Duplicate Names Allowed**
Description: The system incorrectly allows an administrator to save both categories and products with names that are already in use, creating duplicate entries.
Impact: This poses a major threat to the system's data integrity, which will lead to confusion in inventory management, inaccurate reporting, and potential errors in customer orders.
- **B005 (Priority: High) - System Fails to Provide Feedback for Invalid Actions**
Description: Across multiple features (e.g., updating order status non-sequentially), the system correctly blocks the invalid action but fails to display any error message, leaving the user confused as to why the action failed.
Impact: This creates a poor and frustrating user experience for administrators, reducing efficiency and potentially leading them to believe the system is malfunctioning.

For a complete and detailed list of all 15 defects, including their specific reproduction steps and severity, please refer to the file `22127374_BugReport.xlsx`.

9 Self-Assessment

Criteria	Description	Max Points	Self-Assessed Points
Part 1: Data Generation			
2 Tables Selection	2 important tables selected	1.0	1.0
Sample data	All data must be meaningful	2.0	2.0
Data generation report	Report is clear, traceable, professional, with self-assessment	1.0	1.0
Part 2: Scenario Testing			
2 Scenario Selection	2 important scenario selected	1.0	1.0
Scenario testing	Correct and complete scenario identification	2.0	2.0
Use of AI Tools	Prompt transparency, critical validation, added value	1.0	1.0
Test Execution	All designed test cases executed, results logged	0.5	0.5
Bug Reporting	Clear and complete bug report(s), if applicable	0.5	0.5
Scenario testing report	Report is clear, traceable, professional, with self-assessment	1.0	1.0
Total		10.0	10.0